Assignment 11: Docker Image Security Hardening

Objective: Create secure production-ready Docker images by implementing Docker security best practices and vulnerability scanning.

GitHub Repository Structure

assignment11-docker-security/
■■■ app/
■ ■■■ app.py
■■■ Dockerfile
■■■ .dockerignore
■■■ scan-reports/
■ ■■■ trivy-scan.txt
■■■ screenshots/
■■■ docs/
■ ■■■ security-improvements.md
■ ■■■ troubleshooting-guide.md
■ ■■■ deployment-notes.md
■■■ README.md

Assignment Objectives Achieved

✓ Implemented Multi-Stage Docker Build
✓ Configured Non-Root User Execution
✓ Removed Unnecessary Packages
✓ Created .dockerignore File
✓ Performed Vulnerability Scanning using Trivy
✓ Documented Docker Security Best Practices

Dockerfile Security Enhancements

• Used lightweight Python 3.12-slim base image.
• Implemented multi-stage build process.
• Created dedicated non-root user (appuser).
• Reduced attack surface by excluding unnecessary packages.
• Limited container permissions.

.dockerignore Configuration

```
.git
.gitignore
README.md
*.log
__pycache__
.env
```

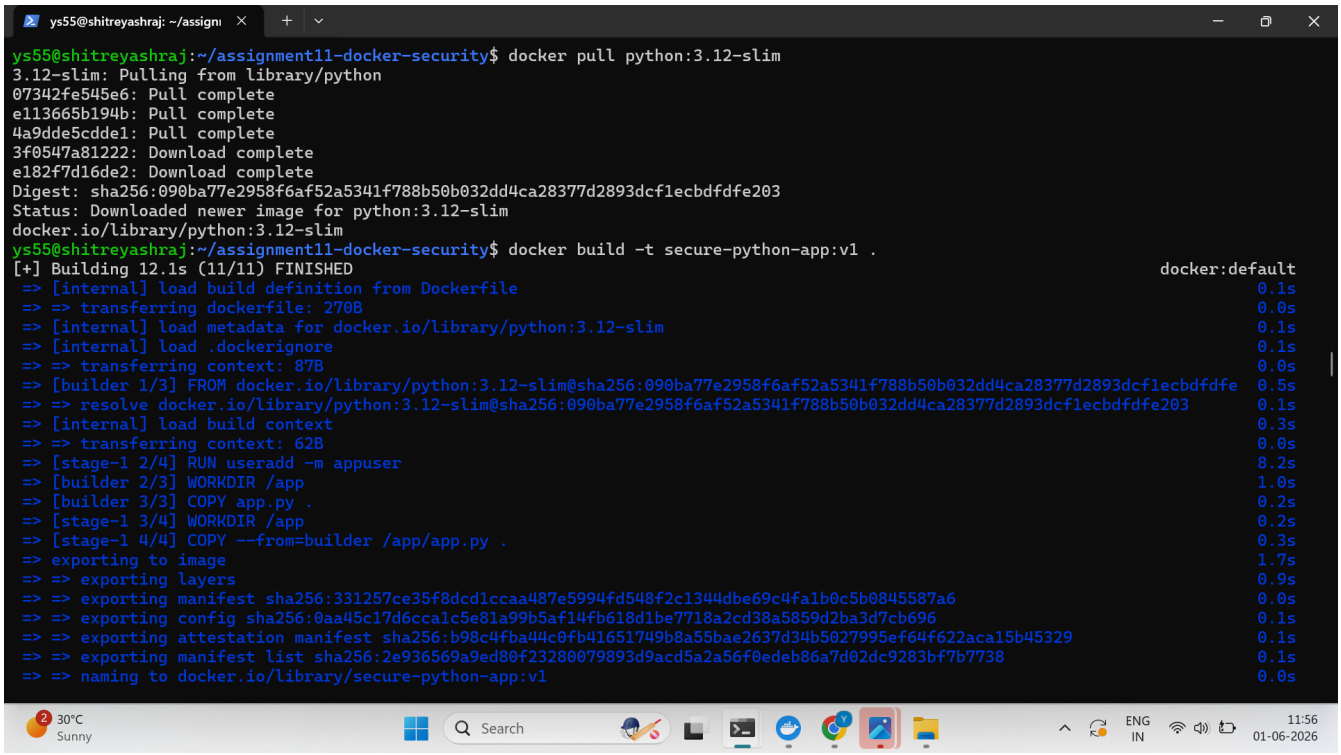
Vulnerability Scanning

Trivy vulnerability scanner was used to analyze the Docker image. The scan verified image security posture and identified potential vulnerabilities. The image was reviewed before deployment to ensure compliance with security standards.

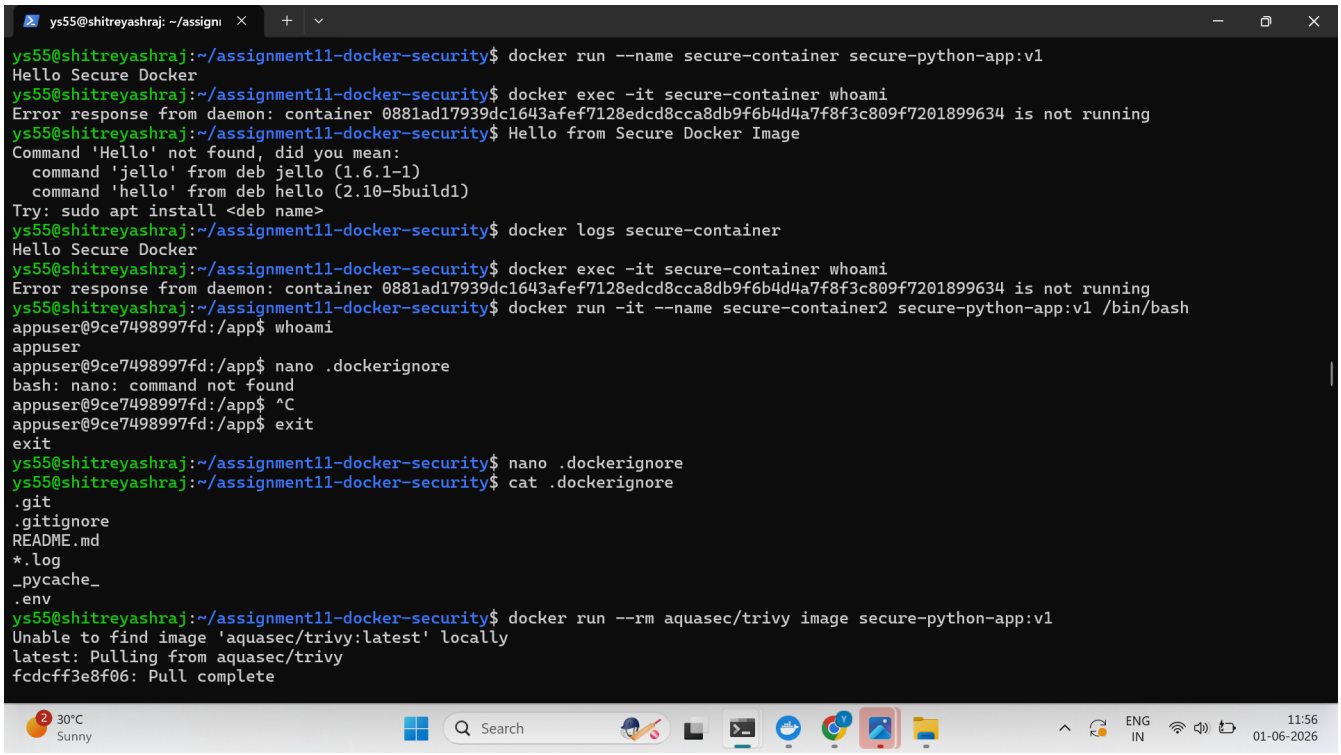
Security Improvements Documentation

1. Non-root container execution.
2. Reduced image size using slim base image.
3. Multi-stage build implementation.
4. Sensitive files excluded through .dockerignore.
5. Vulnerability assessment with Trivy.
6. Production-ready secure Docker image design.

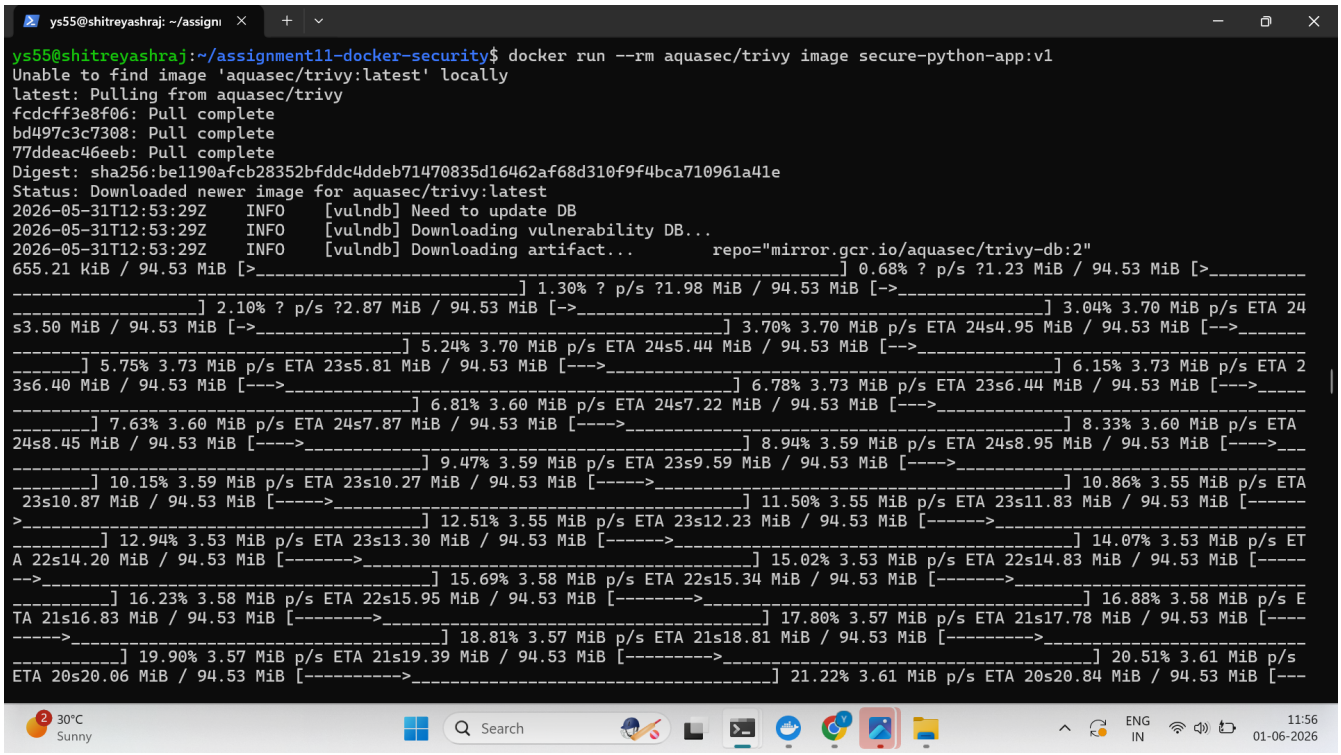
Implementation Evidence



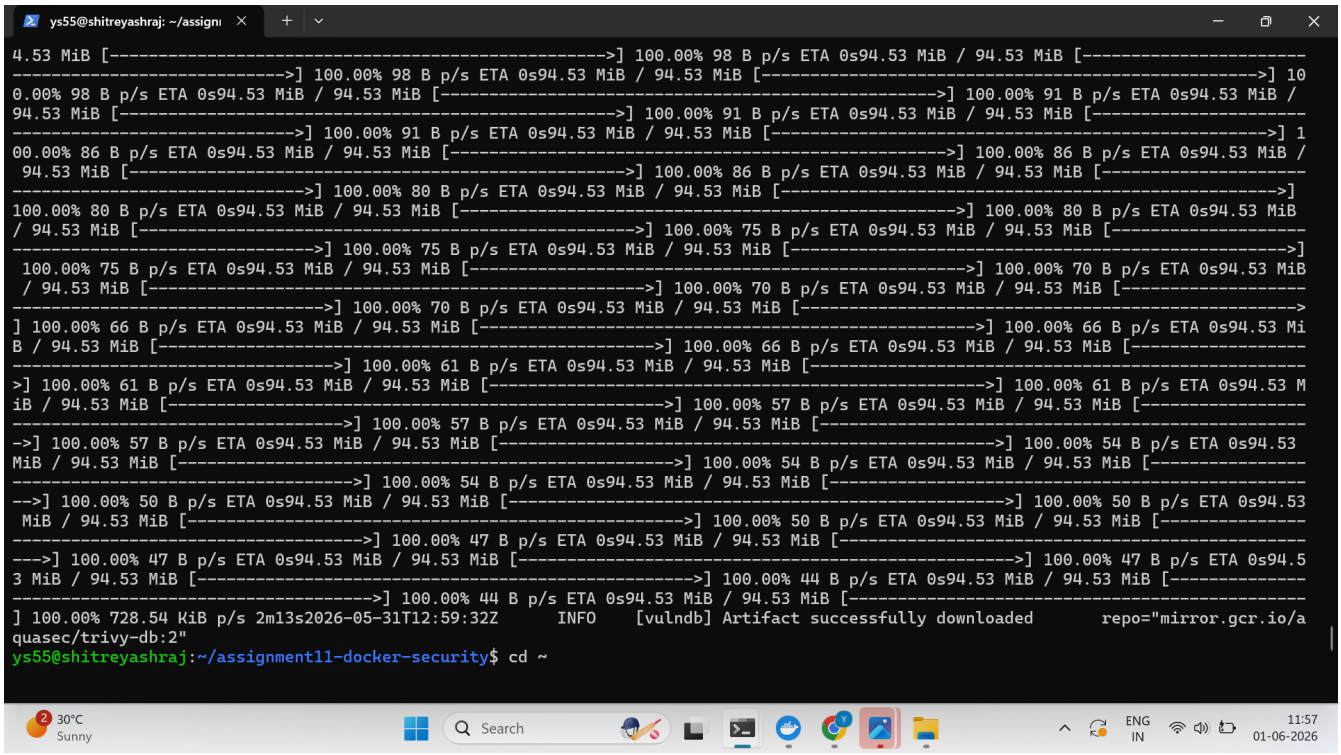
Implementation Evidence



Implementation Evidence



Implementation Evidence



Testing Results

✓ Docker image built successfully using multi-stage build.
✓ Non-root user (appuser) verified inside container.
✓ .dockerignore configuration validated.
✓ Trivy vulnerability scan executed successfully.
✓ Secure Docker image generated and tested.
✓ Security hardening objectives completed.

Conclusion

The Docker image security hardening assignment was completed successfully. Secure image building practices were implemented using multi-stage builds, non-root execution, image optimization, exclusion of unnecessary files, and vulnerability scanning. The resulting image is suitable for production deployment and follows container security best practices.